EC200U&EG91xU Series

QuecOpen(SDK) SPI NOR Flash API Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-09-14

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2021-11-05	Ryan YI	Creation of the document
1.0	2021-11-23	Ryan YI	First official release

1.1	2023-09-14	Sum LI	1. Updated the document name based on the unified QuecOpen naming.
			2. Added applicable modules EG912U-GL and EG915U series.
			3. Added information on SPI NOR flash model adaptation (Chapter 2).
			4. Added 4-wire SPI NOR flash API functions (Chapters 3.1.3.3–3.1.3.6):
			ql_spi_nor_write_8bit_status();
			ql_spi_nor_write_16bit_status();
			ql_spi_nor_read_status_low();
			ql_spi_nor_read_status_high().
			5. Updated ql_errcode_spi_nor_e enumeration of 4-wire SPI NOR flash (Chapter 3.1.3.1.1).
			6. Added 4-wire SPI NOR flash file system mounting API functions (Chapter 3.2):
			ql_spi4_ext_nor_sffs_set_port();
			ql_spi4_ext_nor_sffs_get_port();
			ql_spi4_ext_nor_sffs_mount();
			ql_spi4_ext_nor_sffs_unmount();
			ql_spi4_ext_nor_sffs_mkfs();
			ql_spi4_ext_nor_sffs_is_mount().
			7. Added development example and feature debugging of 4-wire SPI NOR flash file system mounting API (Chapters 4.1.2 and 4.2.2).
			8. Added information on disabling NOR flash write protection (Chapter 4.2.1).

Contents

Table Index

Figure Index

1 Introduction

Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document is applicable to QuecOpen® solution based on CSDK build environment. It outlines external 4-wire SPI NOR flash model adaptation, 4-wire SPI NOR flash API, 4-wire SPI NOR flash file system mounting API, as well as related examples on Quectel EC200U series and EG91xU family modules.

2 SPI NOR Flash Model Adaptation

quec_spi_nor_flash_prop.c and *quec_spi_nor_flash_prop.h*, NOR flash configuration files, are provided in the QuecOpen CSDK, at the path *\components\ql_kernel\drivers*. You can add the specific flash model in configuration files.

2.1. Supported NOR Flash Models

Parameters of some supported external NOR flash models are defined within the *quec_spi_nor_flash_props* in *quec_spi_nor_flash_prop.c*, for example:

```
/* 4-line SPI NOR Flash supported model list,can add by yourself*/
static quec_spi_nor_flash_info_s quec_spi_nor_flash_props[] =
{
//You can cut out it by defining macros
//#define DISABLE_GD25LE64E_C
#ifndef DISABLE_GD25LE64E_C
    {
        //GD25LE64E
        //GD25LE64C
        .mid = 0x1760c8,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
#endif
#ifndef DISABLE_XM25QU64B
    {
        //XM25QU64B
        .mid = 0x175020,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_8BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
#endif
#ifndef DISABLE_XM25QU128C
    {
        //XM25QU128C
        .mid = 0x184120,
        .capacity = 0x1000000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x1000000,
    },
#endif
#ifndef DISABLE_W25Q64JWSSIM
    {
        //W25Q64JWSSIM
        .mid = 0x1780ef,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
#endif
};
```

Figure 1: Parameters of Supported NOR Flash Models

2.2. NOR Flash-Related Parameters

2.2.1. quec_spi_nor_flash_info_s

The *quec_spi_nor_flash_props* (whose type *quec_spi_nor_flash_info_s* is defined in *quec_spi_nor_flash_prop.h*) contains the parameters related to NOR flash model as follows:

```
typedef struct
{
unsigned int mid;
int64    capacity;
      quec_spi_nor_flash_status_mode_e mode;
      unsigned int mount_start_addr;
      int32    mount_size;
}quec_spi_nor_flash_info_s
```

- **Parameter**

Type	Parameter	Description
unsigned int	*mid*	JEDEC ID consisting of Manufacturer ID and Device ID in little-endian format, which is read with **9Fh** command. For example, JEDEC ID = Manufacturer ID (M7–M0) + Device ID (ID15–ID0), in little-endian format: (ID7–ID0) + (ID15–ID8) + (M7–M0).
int64	*capacity*	Capacity size. Unit: byte. For example: for XM25QU32C whose capacity is 32 Mbit, *capacity* is 0x400000; for GD25LE64E whose capacity is 64 Mbit, *capacity* is 0x800000; for XM25QU128C whose capacity is 128 Mbit, *capacity* is 0x1000000.
quec_spi_nor_flash_status_mode_e	*mode*	NOR flash status register type. See ***Chapter 2.2.1.1*** for details.
unsigned int	*mount_start_addr*	Start address of NOR flash when mounting to the file system. 4K alignment is required. It is not of concern if you don't mount NOR flash to the file system.
int32	*mount_size*	Size of partition for mounting NOR flash to the file system. 4K alignment is required. It is not of concern if you don't mount NOR flash to the file system.

2.2.1.1. quec_spi_nor_flash_status_mode_e

The enumeration of NOR flash status register type:

```
typedef enum
{
```

```
    QUEC_NOR_FLASH_MODE_8BIT = 0,
    QUEC_NOR_FLASH_MODE_16BIT,
}quec_spi_nor_flash_status_mode_e
```

- **Member**

Member	Description
QUEC_NOR_FLASH_MODE_8BIT	8-bit status register
QUEC_NOR_FLASH_MODE_16BIT	16-bit or 24-bit status register (For a 24-bit status register, data can only be written to the first 16 bits.)

2.2.1.2. Status Register

The type of status register can be determined with the NOR flash datasheet.

Example 1: As shown in status register definition below in the XM25QU64B datasheet, the status register type is 8-bit.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
	SRWD	QE	BP3	BP2	BP1	BP0	WEL	WIP
Default	0	Note1	0	0	0	0	0	0

Figure 2: XM25QU64B Status Register

Example 2: As shown in the status register definition below in the GD25LQ64E datasheet , two 8-bit status registers form a 16-bit status register, indicating that the status register type is 16-bit.

No.	Name	Description	Note
S7	SRP0	Status Register Protection Bit	Non-volatile writable
S6	BP4	Block Protect Bit	Non-volatile writable
S5	BP3	Block Protect Bit	Non-volatile writable
S4	BP2	Block Protect Bit	Non-volatile writable
S3	BP1	Block Protect Bit	Non-volatile writable
S2	BP0	Block Protect Bit	Non-volatile writable
S1	WEL	Write Enable Latch	Volatile, read only
S0	WIP	Erase/Write In Progress	Volatile, read only
No.	**Name**	**Description**	**Note**
S15	SUS1	Erase Suspend Bit	Volatile, read only
S14	CMP	Complement Protect Bit	Non-volatile writable
S13	LB3	Security Register Lock Bit	Non-volatile writable (OTP)
S12	LB2	Security Register Lock Bit	Non-volatile writable (OTP)
S11	LB1	Security Register Lock Bit	Non-volatile writable (OTP)
S10	SUS2	Program Suspend Bit	Volatile, read only
S9	QE	Quad Enable Bit	Non-volatile writable
S8	SRP1	Status Register Protection Bit	Non-volatile writable

Figure 3: GD25LQ64E Status Register

Example 3: As shown in the instruction set table below in the XM25QU128C datasheet, three 8-bit status registers form a 24-bit status register, indicating that the status register type is 24-bit.

Data Input Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Clock Number	(0 – 7)	(8 – 15)	(16 – 23)	(24 – 31)	(32 – 39)	(40 – 47)	(48 – 55)
Write Enable	06h						
Volatile SR Write Enable	50h						
Write Disable	04h						
Read Status Register-1	05h	(S7-S0)(2)					
Write Status Register-1(4)	01h	(S7-S0)(4)					
Read Status Register-2	35h	(S15-S8)(2)					
Write Status Register-2	31h	(S15-S8)					
Read Status Register-3	15h	(S23-S16)(2)					
Write Status Register-3	11h	(S23-S16)					
Chip Erase	C7h/60h						

Figure 4: XM25QU128C Status Register

3 SPI NOR Flash APIs

3.1. 4-wire SPI NOR Flash API

3.1.1. Header File

ql_api_spi_nor_flash.h, the header file of 4-wire SPI NOR flash API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are all located in this directory.

3.1.2. API Overview

Table 1: API Overview

Function	Description
ql_spi_nor_init()	Initializes SPI (SPI bus and clock frequency only).
ql_spi_nor_init_ext()	Initializes SPI.
ql_spi_nor_write_8bit_status()	Writes data to the 8-bit status register of NOR flash.
ql_spi_nor_write_16bit_status()	Writes data to the 16-bit or 24-bit status register of NOR flash.
ql_spi_nor_read_status_low()	Reads the low 8-bit data of NOR flash status register.
ql_spi_nor_read_status_high()	Reads the high 8-bit data of NOR flash status register.
ql_spi_nor_read()	Reads data from NOR flash.
ql_spi_nor_write()	Writes data to NOR flash.
ql_spi_nor_erase_chip()	Erases the full chip of NOR flash.
ql_spi_nor_erase_sector()	Erases a specified sector (size: 4 KB) of the NOR flash.
ql_spi_nor_erase_64k_block()	Erases a specified block (size: 64 KB) of the NOR flash.

3.1.3. API Description

3.1.3.1. ql_spi_nor_init

This function initializes SPI (SPI bus and clock frequency only). Before calling this function, you need to set relevant GPIO pins to SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

Calling *ql_spi_nor_init()* will configure *ql_spi_input_sel_e* to *QL_SPI_DI_1*, *ql_spi_transfer_mode_e* to *QL_SPI_DIRECT_POLLING*, and *ql_spi_cs_sel_e* to *QL_SPI_CS0* by default. See ***Chapters 3.1.3.1.4***, ***3.1.3.1.5***, and ***3.1.3.1.6*** for details.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_init(ql_spi_port_e port, ql_spi_clk_e spiclk)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

spiclk:
[In] SPI clock frequency. See ***Chapter 3.1.3.1.3*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

Selecting a wrong SPI bus for initialization can lead to failures in reading, writing, or erasing NOR flash. Therefore, you need to check the initialization return value first. If the initialization fails, please do not perform read, write, and erase operations on the NOR flash.

3.1.3.1.1. ql_errcode_spi_nor_e

The enumeration of result codes of 4-wire SPI NOR flash API:

```
typedef ql_errcode_spi_flash_e ql_errcode_spi_nor_e
```

```
typedef enum
{
    QL_SPI_FLASH_SUCCESS                    =    0,
    QL_SPI_FLASH_ERROR                      =    1 |
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
```

```
    QL_SPI_FLASH_PARAM_TYPE_ERROR,
    QL_SPI_FLASH_PARAM_DATA_ERROR,
    QL_SPI_FLASH_PARAM_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_NULL_ERROR,
    QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_LENGTH_ERROR,
    QL_SPI_FLASH_MALLOC_MEM_ERROR,
    QL_SPI_FLASH_NOT_FLASH_CONN_ERROR,
    QL_SPI_FLASH_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_ECC_ERROR,
    QL_SPI_FLASH_PROGRAM_ERROR,
    QL_SPI_FLASH_ERASE_ERROR,
    QL_SPI_FLASH_MUTEX_CREATE_ERROR,
    QL_SPI_FLASH_MUTEX_LOCK_ERROR,
    QL_SPI_FLASH_SET_GPIO_ERROR,
}ql_errcode_spi_flash_e;
```

- **Member**

Member	Description
QL_SPI_FLASH_SUCCESS	Successful execution
QL_SPI_FLASH_ERROR	Other errors related to SPI bus
QL_SPI_FLASH_PARAM_TYPE_ERROR	Parameter type error
QL_SPI_FLASH_PARAM_DATA_ERROR	Parameter data error
QL_SPI_FLASH_PARAM_ACQUIRE_ERROR	Unable to get parameter
QL_SPI_FLASH_PARAM_NULL_ERROR	Parameter NULL error
QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR	Unable to get SPI bus
QL_SPI_FLASH_PARAM_LENGTH_ERROR	Parameter length error
QL_SPI_FLASH_MALLOC_MEM_ERROR	Memory allocation error
QL_SPI_FLASH_NOT_FLASH_CONN_ERROR	Flash chip not connected
QL_SPI_FLASH_NOT_SUPPORT_ERROR	Flash model not supported
QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR	Operation not supported by the flash model
QL_SPI_FLASH_ECC_ERROR	NAND flash ECC error (for external NAND flash only)

QL_SPI_FLASH_PROGRAM_ERROR	Flash program error
QL_SPI_FLASH_ERASE_ERROR	Flash erase error
QL_SPI_FLASH_MUTEX_CREATE_ERROR	Flash mutex creation error
QL_SPI_FLASH_MUTEX_LOCK_ERROR	Flash mutex locking error
QL_SPI_FLASH_SET_GPIO_ERROR	Flash GPIO setting error

3.1.3.1.2. ql_spi_port_e

The enumeration of SPI bus:

```
typedef enum
{
    QL_SPI_PORT1,
    QL_SPI_PORT2,
}ql_spi_port_e;
```

- **Member**

Member	Description
QL_SPI_PORT1	SPI1 bus
QL_SPI_PORT2	SPI2 bus

3.1.3.1.3. ql_spi_clk_e

The enumeration of SPI clock frequency for the module:

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_781_25KHZ = 781250,
    QL_SPI_CLK_1_5625MHZ = 1562500,
    QL_SPI_CLK_3_125MHZ = 3125000,
    QL_SPI_CLK_5MHZ = 5000000,
    QL_SPI_CLK_6_25MHZ = 6250000,
    QL_SPI_CLK_10MHZ = 10000000,
    QL_SPI_CLK_12_5MHZ = 12500000,
    QL_SPI_CLK_20MHZ = 20000000,
    QL_SPI_CLK_25MHZ = 25000000,
```

```
    QL_SPI_CLK_33_33MHZ = 33000000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e;
```

- **Member**

Member	Description
QL_SPI_CLK_INVALID	Invalid parameters
QL_SPI_CLK_781_25KHZ	Clock frequency: 781.25 kHz
QL_SPI_CLK_1_5625MHZ	Clock frequency: 1.5625 MHz
QL_SPI_CLK_3_125MHZ	Clock frequency: 3.125 MHz
QL_SPI_CLK_5MHZ	Clock frequency: 5 MHz
QL_SPI_CLK_6_25MHZ	Clock frequency: 6.25 MHz
QL_SPI_CLK_10MHZ	Clock frequency: 10 MHz
QL_SPI_CLK_12_5MHZ	Clock frequency: 12.5 MHz
QL_SPI_CLK_20MHZ	Clock frequency: 20 MHz
QL_SPI_CLK_25MHZ	Clock frequency: 25 MHz
QL_SPI_CLK_33_33MHZ	Clock frequency: 33.33 MHz
QL_SPI_CLK_50MHZ_MAX	Clock frequency: 50 MHz (Maximum)

3.1.3.1.4. ql_spi_input_sel_e

The enumeration of data input pin:

```
typedef enum
{
    QL_SPI_DI_0 = 0,
    QL_SPI_DI_1,
    QL_SPI_DI_2,
}ql_spi_input_sel_e;
```

- **Member**

Member	Description
QL_SPI_DI_0	DI0 as data input pin (currently not supported)

QL_SPI_DI_1	DI1 as data input pin
QL_SPI_DI_2	DI2 as data input pin (currently not supported)

3.1.3.1.5. ql_spi_transfer_mode_e

The enumeration of SPI transfer mode:

```
typedef enum
{
    QL_SPI_DIRECT_POLLING = 0,
    QL_SPI_DIRECT_IRQ,
    QL_SPI_DMA_POLLING,
    QL_SPI_DMA_IRQ,
}ql_spi_transfer_mode_e;
```

- **Member**

Member	Description
QL_SPI_DIRECT_POLLING	FIFO read/write. Polling waiting.
QL_SPI_DIRECT_IRQ	FIFO read/write. Interrupt notification. Currently not supported.
QL_SPI_DMA_POLLING	DMA read/write. Polling waiting.
QL_SPI_DMA_IRQ	DMA read/write. Interrupt notification. Currently not supported.

NOTE

When the module uses DMA polling mode, SPI and SD card preempt DMA channels. If the SD card already uses DMA channels, SPI initialization will fail.

3.1.3.1.6. ql_spi_cs_sel_e

The enumeration of CS pin:

```
typedef enum
{
    QL_SPI_CS0 = 0,
    QL_SPI_CS1,
    QL_SPI_CS2,
    QL_SPI_CS3,
}ql_spi_cs_sel_e;
```

- **Member**

Member	Description
QL_SPI_CS0	CS0 as SPI CS pin
QL_SPI_CS1	CS1 as SPI CS pin
QL_SPI_CS2	CS2 as SPI CS pin (currently not supported)
QL_SPI_CS3	CS3 as SPI CS pin (currently not supported)

3.1.3.2. ql_spi_nor_init_ext

This function initializes SPI, including SPI bus, clock frequency, transfer mode, data input pin, and CS pin. Before calling this function, you need to set relevant GPIO pins to SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_init_ext(ql_spi_nor_config_s nor_config)
```

- **Parameter**

nor_config:
[In] SPI configuration parameters. See ***Chapter 3.1.3.2.1*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

Selecting a wrong SPI bus for initialization can lead to failures in reading, writing, or erasing NOR flash. Therefore, you need to check the initialization return value first. If the initialization fails, please do not perform read, write, and erase operations on NOR flash.

3.1.3.2.1. ql_spi_nor_config_s

The structure of SPI configuration parameters:

```
typedef ql_spi_flash_config_s ql_spi_nor_config_s
```

```
typedef struct
```

```
{
    ql_spi_port_e port;
    ql_spi_clk_e spiclk;
    ql_spi_input_sel_e input_sel;
    ql_spi_transfer_mode_e transmode;
    ql_spi_cs_sel_e cs;
} ql_spi_flash_config_s;
```

- **Parameter**

Type	Parameter	Description
ql_spi_port_e	*port*	SPI bus. See ***Chapter 3.1.3.1.2*** for details.
ql_spi_clk_e	*spiclk*	SPI clock frequency. See ***Chapter 3.1.3.1.3*** for details.
ql_spi_input_sel_e	*input_sel*	Data input pin. See ***Chapter 3.1.3.1.4*** for details.
ql_spi_transfer_mode_e	*transmode*	SPI transfer mode. See ***Chapter 3.1.3.1.5*** for details.
ql_spi_cs_sel_e	*cs*	CS pin. See ***Chapter 3.1.3.1.6*** for details.

3.1.3.3. ql_spi_nor_write_8bit_status

This function writes data to the 8-bit status register of NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_write_8bit_status(ql_spi_port_e port, unsigned char status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

status:
[In] Data to be written to the 8-bit status register of NOR flash.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.4. ql_spi_nor_write_16bit_status

This function writes data to the 16-bit or 24-bit status register of NOR flash. For a 24-bit status register, data can only be written to the first 16 bits.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_write_16bit_status(ql_spi_port_e port, unsigned short status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

status:
[In] Data to be written to the 16-bit status register of NOR flash.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.5. ql_spi_nor_read_status_low

This function reads the low 8-bit data from NOR flash status register.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_read_status_low(ql_spi_port_e port, unsigned char *status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

status:
[Out] Low 8-bit data read from the NOR flash status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.6. ql_spi_nor_read_status_high

This function reads the high 8-bit data from NOR flash status register.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_read_status_high(ql_spi_port_e port, unsigned char *status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

status:
[Out] High 8-bit data read from the NOR flash status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.7. ql_spi_nor_read

This function reads data from NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_read(ql_spi_port_e port, unsigned char* data, unsigned int addr, unsigned short len)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

data:
[Out] Pointer to the read data.

addr:
[In] Address for storing the data to be read.

len:
[In] Length of the data to be read. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.8. ql_spi_nor_write

This function writes data to NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_write(ql_spi_port_e port, unsigned char *data, unsigned int addr, unsigned short len)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

data:
[In] Data to be written.

addr:
[In] Address for storing the data to be written.

len:
[In] Length of the data to be written. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.9. ql_spi_nor_erase_chip

This function erases the full chip of NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_erase_chip(ql_spi_port_e port)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.10. ql_spi_nor_erase_sector

This function erases a specified sector (size: 4 KB) of the NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_erase_sector(ql_spi_port_e port, unsigned int addr)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

addr:
[In] Address of the sector to be erased.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.11. ql_spi_nor_erase_64k_block

This function erases a specified block (size: 64 KB) of the NOR flash.

- **Prototype**

```
ql_errcode_spi_nor_e ql_spi_nor_erase_64k_block(ql_spi_port_e port, unsigned int addr)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

addr:
[In] Address of the block to be erased.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.2. 4-wire SPI NOR Flash File System Mounting API

3.2.1. Header File

ql_api_spi4_ext_nor_sffs.h, the header file of 4-wire SPI NOR flash file system mounting API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, the header files mentioned in this document are all located in this directory.

3.2.2. API Overview

Table 2: API Overview

Function	Description
ql_spi4_ext_nor_sffs_set_port()	Sets the SPI bus used in the SFFS file system.
ql_spi4_ext_nor_sffs_get_port()	Gets the SPI bus used in the SFFS file system.
ql_spi4_ext_nor_sffs_mount()	Mounts the 4-wire SPI NOR flash to the SFFS file system.
ql_spi4_ext_nor_sffs_unmount()	Unmounts the 4-wire SPI NOR flash from the SFFS file system.
ql_spi4_ext_nor_sffs_mkfs()	Formats the 4-wire SPI NOR flash to the SFFS format.
ql_spi4_ext_nor_sffs_is_mount()	Gets whether the 4-wire SPI NOR flash is mounted to the SFFS file system.

3.2.3. API Description

3.2.3.1. ql_spi4_ext_nor_sffs_set_port

This function sets the SPI bus used in the SFFS file system. After initializing the 4-wire SPI NOR flash, you must call this interface to set the SPI port currently in use, otherwise, *QL_SPI_PORT1* will be used by default.

- **Prototype**

```
void ql_spi4_ext_nor_sffs_set_port(ql_spi_port_e spi_port)
```

- **Parameter**

port:
[In] SPI bus. See ***document [3]*** for details.

- **Return Value**

None

3.2.3.2. ql_spi4_ext_nor_sffs_get_port

This function gets the SPI bus used in the SFFS file system.

- **Prototype**

```
ql_spi_port_e ql_spi4_ext_nor_sffs_get_port(void)
```

- **Parameter**

None

- **Return Value**

SPI bus. See ***document [3]*** for details.

3.2.3.3. ql_spi4_ext_nor_sffs_mount

This function mounts the 4-wire SPI NOR flash to the SFFS file system.

- **Prototype**

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_mount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.3.1*** for details.

3.2.3.3.1. ql_errcode_spi4_extnsffs_e

The enumeration of result codes of 4-wire SPI NOR flash file system mounting:

```
typedef enum
{
    QL_SPI4_EXT_NOR_SFFS_SUCCESS    =     0,
    QL_SPI4_EXT_NOR_SFFS_NOT_INIT_ERROR =     100 |
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
    QL_SPI4_EXT_NOR_SFFS_CREATE_FBDEV2SPI_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_MOUNT_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_UNMOUNT_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_MKFS_ERROR,
```

```
    QL_SPI4_EXT_NOR_SFFS_LOCK_ERROR,
}ql_errcode_spi4_extnsffs_e
```

- **Member**

Member	Description
QL_SPI4_EXT_NOR_SFFS_SUCCESS	Successful execution
QL_SPI4_EXT_NOR_SFFS_NOT_INIT_ERROR	SPI NOR flash initialization error
QL_SPI4_EXT_NOR_SFFS_CREATE_FBDEV2SPI_ERROR	Block device partition creation error
QL_SPI4_EXT_NOR_SFFS_SFFS_MOUNT_ERROR	Mounting error
QL_SPI4_EXT_NOR_SFFS_SFFS_UNMOUNT_ERROR	Unmounting error
QL_SPI4_EXT_NOR_SFFS_SFFS_MKFS_ERROR	Formatting error
QL_SPI4_EXT_NOR_SFFS_LOCK_ERROR	Mutex locking error

3.2.3.4. ql_spi4_ext_nor_sffs_unmount

This function unmounts 4-wire SPI NOR flash from the SFFS file system.

- **Prototype**

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_unmount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.3.1*** for details.

3.2.3.5. ql_spi4_ext_nor_sffs_mkfs

This function formats the 4-wire SPI NOR flash to the SFFS format.

- **Prototype**

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_mkfs(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.3.1*** for details.

3.2.3.6. ql_spi4_ext_nor_sffs_is_mount

This function gets whether the 4-wire SPI NOR flash is mounted to the SFFS file system.

- **Prototype**

```
ql_spi4_extnsffs_status_e ql_spi4_ext_nor_sffs_is_mount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.6.1*** for details.

3.2.3.6.1. ql_spi4_extnsffs_status_e

The enumeration of 4-wire SPI NOR flash status:

```
typedef enum
{
    QL_SPI4_EXT_NOR_SFFS_IS_UNMOUNTED   =   0,
    QL_SPI4_EXT_NOR_SFFS_IS_MOUNTED
}ql_spi4_extnsffs_status_e
```

- **Member**

Member	Description
QL_SPI4_EXT_NOR_SFFS_IS_UNMOUNTED	SPI NOR flash is not mounted to the file system.
QL_SPI4_EXT_NOR_SFFS_IS_MOUNTED	SPI NOR flash is mounted to the file system.

4 Example

This chapter introduces how to use the above APIs at the application layer for development and simple debugging of 4-wire SPI NOR flash and SPI NOR flash file system mounting.

4.1. Development Example

4.1.1. 4-wire SPI NOR Flash API Development

\components\ql-application\spi_nor_flash\spi_nor_flash_demo.c, the demo file for 4-wire SPI NOR flash API, is provided in QuecOpen CSDK of the module. It includes:

- SPI NOR flash initialization
- Reading and writing data
- Erasing a sector
- Erasing a block
- Erasing the full chip

The entry function is *ql_spi_nor_flash_demo_init()*, as shown below:

```
QlOSStatus ql_spi_nor_flash_demo_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&spi_nor_flash_demo_task, SPI_NOR_FLASH_DEMO_TASK_STACK_SIZE, SPI_NOR_FLASH_DEMO_TASK_PRIO, "ql_spi_nor", ql_spi_nor_flash_demo_task_pthread, NULL, SPI_NOR_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI_NOR_LOG("demo_task created failed");
        return err;
    }

    return err;
}
```

Figure 5: Entry Function: *ql_spi_nor_flash_demo_init()*

As shown below, the above example is disabled by default in *ql_init_demo_thread*. Uncomment *ql_spi_nor_flash_demo_init()* if you need to run it.

```
#ifdef QL_APP_FEATURE_SPI_NOR_FLASH
    //ql_spi_nor_flash_demo_init();
#endif
```

Figure 6: SPI NOR Flash API Example Disabled by Default

4.1.2. 4-wire SPI NOR Flash File System Mounting API Development

\components\ql-application\spi4_ext_nor_sffs\spi4_ext_nor_sffs_demo.c, the demo file for 4-wire SPI NOR flash file system mounting API, is provided in QuecOpen CSDK of the module. It includes:

- SPI NOR flash initialization
- Mounting SFFS file system
- Formatting SPI NOR flash
- Read and write data, unmount from the SFFS file system through the file system API, etc.

The entry function is *ql_spi4_ext_nor_sffs_demo_init()*, as shown below:

```
QlOSStatus ql_spi4_ext_nor_sffs_demo_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    //QL_SPI4_EXTNSFFS_LOG("enter ql_spi4_ext_nor_sffs_demo_init");
    err = ql_rtos_task_create(&spi4_ext_nor_sffs_demo_task,
                              SPI4_EXT_NOR_SFFS_DEMO_TASK_STACK_SIZE,
                              SPI4_EXT_NOR_SFFS_DEMO_TASK_PRIO,
                              "ql_spi4_sffs",
                              ql_spi4_ext_nor_sffs_demo_task_pthread,
                              NULL,
                              SPI4_EXT_NOR_SFFS_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI4_EXTNSFFS_LOG("demo_task created failed");
        return err;
    }

    return err;
}» end ql_spi4_ext_nor_sffs_demo_init »
```

Figure 7: Entry Function: *ql_spi4_ext_nor_sffs_demo_init()*

As shown below, the above example is disabled by default in *ql_init_demo_thread*. Uncomment *ql_spi4_ext_nor_sffs_demo_init()* if you need to run it.

```
#ifdef QL_APP_FEATURE_SPI4_EXT_NOR_SFFS
    //ql_spi4_ext_nor_sffs_demo_init();
#endif
```

Figure 8: 4-wire SPI NOR Flash File System Mounting API Example Disabled by Default

4.2. Feature Debugging

4.2.1. 4-wire SPI NOR Flash API

Before debugging the 4-wire SPI NOR flash feature, it is necessary to connect a NOR flash chip to LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to a PC with a USB cable. USB AP Log Port primarily displays system debugging information. You can view the example through USB AP Log Port after capturing the log with cooltools. For details about log capture, see ***document [4]***.

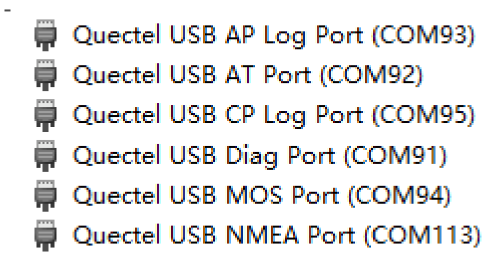
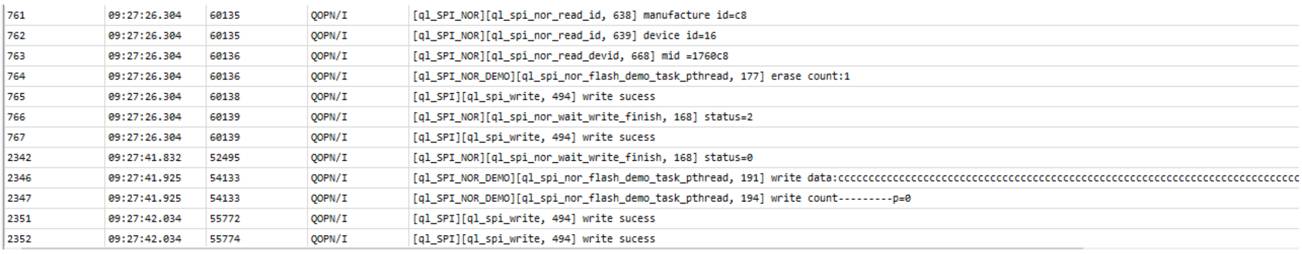


Figure 9: Ports in Device Manager

ql_spi_nor_flash_demo_init() runs automatically after the module is turned on. Determine whether the read and write operations are successful by checking the consistency of the read and written data. Read data after erase operations, which can be identified as successful if all the read data are 'FF'.

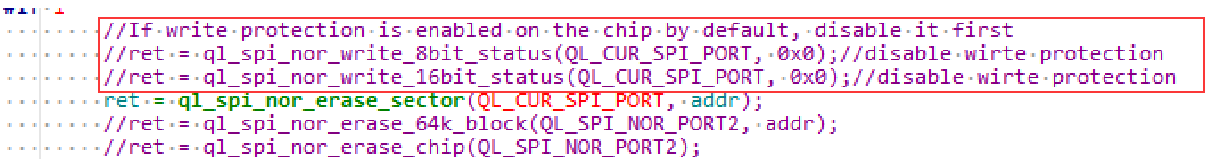
In the case of a failed execution, you can view the reason for the failure as per the corresponding log information.



761	09:27:26.304	60135	QOPN/I	[ql_SPI_NOR][ql_spi_nor_read_id, 638] manufacture id=c8
762	09:27:26.304	60135	QOPN/I	[ql_SPI_NOR][ql_spi_nor_read_id, 639] device id=16
763	09:27:26.304	60136	QOPN/I	[ql_SPI_NOR][ql_spi_nor_read_devid, 668] mid =1760c8
764	09:27:26.304	60136	QOPN/I	[ql_SPI_NOR_DEMO][ql_spi_nor_flash_demo_task_pthread, 177] erase count:1
765	09:27:26.304	60138	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
766	09:27:26.304	60139	QOPN/I	[ql_SPI_NOR][ql_spi_nor_wait_write_finish, 168] status=2
767	09:27:26.304	60139	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
2342	09:27:41.832	52495	QOPN/I	[ql_SPI_NOR][ql_spi_nor_wait_write_finish, 168] status=0
2346	09:27:41.925	54133	QOPN/I	[ql_SPI_NOR_DEMO][ql_spi_nor_flash_demo_task_pthread, 191] write data:cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
2347	09:27:41.925	54133	QOPN/I	[ql_SPI_NOR_DEMO][ql_spi_nor_flash_demo_task_pthread, 194] write count---------p=0
2351	09:27:42.034	55772	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
2352	09:27:42.034	55774	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess

Figure 10: Log Information

If the NOR flash is write-protected, this causes the API to execute successfully without actually erasing or writing data. Select the specified function to clear the write-protected bit(s) of the status register according to the value of *mode* in the *quec_spi_nor_flash_info_s* structure, as shown in the following figure.



```
//If write protection is enabled on the chip by default, disable it first
//ret = ql_spi_nor_write_8bit_status(QL_CUR_SPI_PORT, 0x0);//disable wirte protection
//ret = ql_spi_nor_write_16bit_status(QL_CUR_SPI_PORT, 0x0);//disable wirte protection
ret = ql_spi_nor_erase_sector(QL_CUR_SPI_PORT, addr);
//ret = ql_spi_nor_erase_64k_block(QL_SPI_NOR_PORT2, addr);
//ret = ql_spi_nor_erase_chip(QL_SPI_NOR_PORT2);
```

Figure 11: Disable NOR Flash Write Protection

4.2.2. 4-wire SPI NOR Flash File System Mounting API

Before debugging the SPI NOR flash file system mounting feature, it is necessary to connect a NOR flash to LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to a PC with a USB cable. USB AP Log Port mainly displays system debugging information. You can view the

example through USB AP Port after capturing the log with cooltools. For details about log capture, see ***document [4]***. For ports in device manager of the module, see ***Figure 9***.

ql_spi4_ext_nor_sffs_demo_init() runs automatically after the module is turned on. Through the log, you can see the information about the 4-wire SPI NOR flash file system mounting. The following figure shows the debugging information of the AP log port.

```
[16:47:51.142]  [19877] [ 1169] [               ] [      ] [D] FSYS/I : FBDEV: scan EB quickinit/0 invalid/1 samelb/0 erase count/3/3 next use/20
[16:47:51.142]  [19877] [ 1170] [               ] [      ] [D] FSYS/I : SFFS mount, device block size/500 block count/16255 cache count 4
[16:47:51.168]  [20038] [ 1171] [               ] [      ] [D] FSYS/I : SFFS mount move count/0 block free/8067
[16:47:51.168]  [20337] [ 1172] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 160] sffs mount succeed
[16:47:51.168]  [20343] [ 1173] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 170] fs free_size :3964892
[16:47:51.168]  [20345] [ 1174] [               ] [      ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:51.233]  [20372] [ 1175] [               ] [      ] [ ] FSYS/I : prvFlashErase,offset=0xa0000,size=0x8000, phy_size=0x7f8000
[16:47:51.510]  [23439] [ 1494] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fopen, 169] open file EXNSFFS:test.txt, fd 5123
[16:47:51.510]  [23445] [ 1495] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fseek, 438] ret:0
[16:47:51.510]  [23448] [ 1496] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 204] file read result is 1234567890abcdefg
[16:47:51.516]  [25022] [ 1503] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fclose, 264] close file fd 5123
[16:47:51.516]  [25023] [ 1504] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 212] sffs unmount succeed
[16:47:51.516]  [25026] [ 1505] [               ] [      ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:51.516]  [25028] [ 1506] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fopen, 119] get free size failed
[16:47:51.516]  [25028] [ 1507] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 216] open file failed,errcode is 830301a3
[16:47:51.589]  [25029] [ 1508] [               ] [      ] [D] FSYS/I : FBDEV: create phys_start/0x0 phys_size/0x7f8000 eb_size/0x8000 pb_size/0x200 read_only/0
[16:47:56.085]  [35466] [ 1910] [               ] [      ] [D] FSYS/I : FBDEV: scan EB quickinit/0 invalid/1 samelb/0 erase count/3/3 next use/148
[16:47:56.108]  [35466] [ 1911] [               ] [      ] [D] FSYS/I : SFFS mount, device block size/500 block count/16255 cache count 4
[16:47:56.108]  [35631] [ 1912] [               ] [      ] [D] FSYS/I : SFFS mount move count/0 block free/8066
[16:47:56.108]  [35753] [ 1913] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 225] sffs remount succeed
[16:47:56.108]  [35758] [ 1914] [               ] [      ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:56.217]  [37317] [ 1921] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fopen, 169] open file EXNSFFS:test.txt, fd 5123
[16:47:56.217]  [37319] [ 1922] [               ] [      ] [ ] QOPN/I : [ql_fs][ql_fclose, 264] close file fd 5123
[16:47:56.217]  [37319] [ 1923] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 236] close file
[16:47:56.217]  [37320] [ 1924] [               ] [      ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 249] ql_rtos_task_delete
[16:47:56.267]  [37321] [ 1925] [               ] [      ] [ ] QOPN/I : [ql_OSI][ql_rtos_task_delete, 183] remove task 80C0B3A0 ok
```

Figure 12: Log information on 4-wire SPI NOR Flash File System Mounting

When formatting SPI NOR flash, if the dump log shown in the figure below appears, it may be because NOR flash is write-protected, resulting in data not being written to NOR flash normally.

```
11:48:21.128    7387        QOPN/I      [ql_SPI_NOR][ql_spi_nor_read_id, 622] device id=17
11:48:21.128    7388        QOPN/I      [ql_SPI_NOR][ql_spi_nor_read_devid, 651] mid =184120
11:48:21.128    7389        QUEV/I      [ql_SPI4SFFS][drvGeneralSpiFlashOpen, 144] Flash 0x0,0x400000,0x1000000
11:48:21.128    7389        FSYS/I      FBDEV: create phys_start/0x0 phys_size/0x3ff000 eb_size/0x1000 pb_size/0x200 read_only/0
11:48:21.128    7604        QOPN/I      [ql_LEDCFGDEMO][ql_ledcfg_demo_thread, 165] led config slow twinkle [0]
11:48:21.128    7604        QOPN/I      [ql_PWM][ql_pwm_lpg_enable, 147] LPG enable OK!
11:48:21.209    7604        QOPN/I      [ql_LEDCFGDEMO][ql_ledcfg_demo_thread, 200] led config network is not 4G
11:48:21.338    11908       FSYS/I      FBDEV: scan EB quickinit/0 invalid/1023 samelb/0 erase count/15728640/0 next use/0
11:48:21.338    11909       FSYS/I      SFFS mount, device block size/500 block count/8175 cache count 4
11:48:21.338    11910       FSYS/E      SFFS mount wrong root tag
11:48:21.338    11910       FSYS/I      prvMountSffs mount failed!
11:48:21.393    11911       FSYS/I      FBDEV: create phys_start/0x0 phys_size/0x3ff000 eb_size/0x1000 pb_size/0x200 read_only/0
11:48:21.604                            Detected event: 0x00a55e47
11:48:21.604                            Detected event: 0x9db10000
11:48:21.712    16417       FSYS/I      FBDEV: scan EB quickinit/0 invalid/1023 samelb/0 erase count/15728640/0 next use/0
11:48:21.822                            Detected event: 0x9db00000
11:48:22.052    16536       FSYS/I      prvFlashErase,offset=0x0,size=0x1000, phy_size=0x3ff000
11:48:22.052    16537       QUEV/I      [ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052    16539       QUEV/I      [ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052    16539       QUEC/I      drvGeneralSpiFlashErase Success
11:48:22.052    16539       QUEC/I      drvGeneralSpiFlashErase 0x1000,0x0
11:48:22.052    16547       FSYS/I      FBDEV: prvFlash Write Enter, offset=0x0,  data=0x80bfd6b8, size=0x200, phy_size=0x3ff000
11:48:22.052    16550       QUEV/I      [ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052    16552       QUEV/I      [ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052    16552       QUEC/I      drvGeneralSpiFlashWrite 0x200,0x0
11:48:22.052    16553       FSYS/I      FBDEV: prvFlash Write Exit
11:48:22.052    16553       FSYS/I      prvFlashWriteCheck, offset=0x0,data=0x80bfd6b8,size=0x200,
11:48:22.052    16557       KERN/E      !!! PANIC AT 6009AAAB
11:48:22.052    16558       IPCD/E      !!! CP PANIC:
11:48:22.052    16561           /E      !!! BLUE SCREEN pc/0x800b6a lr/0x800b6b sp/0x80bfcf30 cpsr/0x200701db spsr/0x2007003f
11:48:22.052    23194       KERN/I      TASK count 58
11:48:22.052    23194       KERN/I      TASK 58 (ql_spi4_sffs) state/0 prio/12/12
```

failed to mount

Figure 13: Dump Log

At this time, select the specified function to clear the write-protected bit(s) of the status register according to the value of *mode* in the *quec_spi_nor_flash_info_s* structure.

5 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_GPIO_API_Reference_Manual
[3] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_SPI_Development_Guide
[4] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CS	Chip Select
DI	Data Input
DMA	Direct Memory Access
ECC	Error Correcting Code
EVB	Evaluation Board
FIFO	First In First Out
GPIO	General-Purpose Input/Output
ID	Identifier
IoT	Internet of Things
JEDEC	Joint Electron Device Engineering Council

LTE	Long-Term Evolution
PC	Personal Computer
RTOS	Real-Time Operating System
SD	Secure Digital Card
SDK	Software Development Kit
SPI	Serial Peripheral Interface
USB	Universal Serial Bus